dbatools MCP: HTTPS deployment and remote SQL validation

Implementation record and operating guide

29 October 2026 | Asia/Kolkata (IST)

Contents

- Purpose and completion status
- Architecture and environment
- Transport, authentication, and authorization
- Windows runtime and repository preparation
- Create and configure the POC HTTPS certificate
- Start HTTPS and verify the network path
- Login and MCP Inspector testing
- Validate SQL on the MCP host
- Connect to the second SQL Server
- Authorization test and serialization fixes
- Operations, restarts, and troubleshooting
- Claude Agent SDK application handoff
- Verification record and implementation map

Purpose and completion status

This guide records the work completed on the dbatools MCP server, the actual Windows/Linux proof of concept, and the steps needed to operate and reproduce it. It covers the server repository, not the separate Claude Agent SDK application's implementation.

Repository: https://github.com/jagadd/dbatools
Working branch: `development`
Application version observed: `0.8.0`
Implementation baseline for this guide: `7c2b17d905c51bc18fc2e63b8603387167199515`

All changes are made and pushed to `development`. The owner creates the development-to-main pull request manually. The existing `AGENTS.md` contains project context and contributor instructions; the owner decides whether to include it when merging.

What is verified

- Existing STDIO support is retained, and Windows HTTPS Streamable HTTP is added.
- Linux successfully connected to the Windows endpoint with certificate verification.
- Local Windows account login and bearer authentication worked in MCP Inspector.
- PowerShell execution identity and 731 indexed dbatools commands were confirmed.
- A local SQL version query returned `17.0.1125.2`.
- The MCP host reached the second SQL Server on TCP 1433 and used a stored SQL credential.
- A restricted SQL login encountered an authorization failure; after promotion to sysadmin the same query succeeded without changing the MCP credential profile.
- Query output normalization was verified against the remote SQL Server without `selectProperties`: only the two requested columns were returned.
- At the implementation baseline, 18 automated tests, type checking, packaging checks, and native Windows CI acceptance tests passed.

What is still separate or pending

- The Linux Claude Agent SDK application exists separately. Its final end-to-end Foundry/chat/incident workflow is not established by the MCP tests in this guide.
- Automatic Windows service startup, production certificate lifecycle management, enterprise OAuth, unattended token renewal, and per-change application approvals are not claimed as completed.
- The SQL checks performed are connectivity and selected operational checks, not a comprehensive SQL Server health assessment.

No passwords, bearer tokens, private keys, or encrypted credential contents are included here. Hostnames, private addresses, usernames, and paths below describe this POC and must be adapted for another environment.

Architecture and environment

```
Linux app / MCP Inspector
          |
          | HTTPS + Authorization: Bearer <session token>
          v
Windows MCP host: 10.10.0.5:8443/mcp
          |
          | Node.js / TypeScript -> fixed PowerShell driver -> dbatools
          |
          +--> Local SQL Server: localhost
          |    Windows authentication as the MCP process account
          |
          +--> Remote SQL Server: 10.70.1.5,1433
               SQL authentication from an authorized credential reference
```

Component	Observed configuration
Windows MCP host	`2276259-pyhome`, private IP `10.10.0.5`
MCP execution account	`2276259-pyhome\jaga_admin`
MCP repository	`C:\Users\jaga_admin\dbatools`
MCP endpoint	`https://10.10.0.5:8443/mcp`
Linux app host	`2276259dbapyvm`, user `jaga_admin`
Linux trust certificate	`/home/jaga_admin/mcp-certs/server-cert.pem`
Remote SQL host	`windowstest`, private IP `10.70.1.5`
Remote SQL endpoint	`10.70.1.5,1433`, default `MSSQLSERVER` instance
Remote credential alias	`windowstest-reader`
Remote SQL login proposed for POC	`mcp_poc_reader`; use the actual login saved in the profile
Windows runtime observed	Node.js `24.19.0`, PowerShell `7.6.9`
dbatools observed	`2.8.2`, with 731 commands indexed

Both Windows machines are not domain joined. Local accounts with matching names on two machines are separate Windows identities. They do not automatically provide cross-machine integrated authentication.

Why Node.js, npm, and PowerShell are all present

The MCP wrapper is TypeScript compiled to JavaScript and executed by Node.js. PowerShell runs the dbatools commands; SQL Server performs the database work.

Command	Meaning
`npm ci`	Install the exact Node dependency versions in `package-lock.json`
`npm run build`	Compile TypeScript into JavaScript in `dist/`
`npm run refresh-help`	Run PowerShell to generate the local dbatools command index
`node dist/server.js`	Start the built server with the selected transport
`npm test`	Build and run automated protocol and behavior checks

Python is not required to run the server. A Python rewrite was considered and deferred because it would not improve connectivity or remove authentication and certificate requirements. The PDF regeneration utility uses Python only as a documentation build tool.

Use a PowerShell 7 console or VS Code configured for PowerShell 7. Windows PowerShell ISE uses Windows PowerShell 5.1 and does not host PowerShell 7 correctly. Launching interactive `pwsh` inside ISE can leave ISE showing a running script.

Transport, authentication, and authorization

STDIO and HTTPS

`DBATOOLS_TRANSPORT=stdio` is the default. A local client launches the process and communicates over stdin/stdout. STDIO does not expose an HTTP listener or require an additional MCP login. PowerShell runs as the launching Windows account.

`DBATOOLS_TRANSPORT=https` exposes `/mcp` using stateless Streamable HTTP. This is not the older HTTP+SSE transport. The server uses the same four tools and help resources for both transports.

HTTPS starts a fresh PowerShell process for each execution and permits two concurrent executions, including environment probes. STDIO can use a persistent PowerShell process with a serialized request queue. A failed or timed-out HTTPS call must not terminate another caller's process.

MCP tool	Purpose
`list_dbatools_commands`	Discover commands by verb, noun, keyword, and risk
`get_dbatools_command_help`	Inspect parameters, aliases, help, and examples
`invoke_dbatools_command`	Invoke an indexed command with validated arguments
`check_dbatools_environment`	Check runtime, module, index, and execution identity

Help resources are exposed as `dbatools://command/{name}`. The server advertises four generic MCP tools; it does not advertise each dbatools command as an individual MCP tool.

Three different identities and secrets

Layer	Purpose
HTTPS certificate	Encrypt transport and establish the MCP server's identity
MCP Windows login / bearer session	Authenticate and authorize the remote caller
MCP execution account	Windows identity under which PowerShell runs
SQL credential profile	Optional separate identity used to connect to a SQL target

MCP sign-in does not impersonate the user. In this POC both the execution account and the account used for MCP sign-in happen to be `jaga_admin`; these are still different responsibilities.

Windows login and sessions

`POST /auth/login` validates a local Windows account through a fixed PowerShell helper using Windows logon APIs. Names must be `SERVER\username`, where SERVER is the MCP host computer name. Domain login and bare usernames are outside this POC.

- `McpReaders`: discovery, help, resources, and environment checks.
- `McpOperators`: those capabilities plus command invocation.
- SQL permissions are separately enforced by the SQL Server target.

Tokens are random, opaque, 43-character Base64URL values. The server stores token hashes and associated SID/roles in memory, not the plaintext tokens.

- Lifetime: one hour from issue; no automatic refresh mechanism is implemented.
- Any MCP process restart invalidates all existing sessions immediately.
- `POST /auth/logout` revokes the current token.
- All MCP requests require `Authorization: Bearer <token>`.
- Tokens are not bound to Inspector and can be used by another authorized client.
- Token length does not indicate freshness. Expiration or restart explains many apparently well-formed but rejected tokens.
- Login is limited to five attempts per source address per 15 minutes; Windows account lockout policy also applies.

The current login is a custom POC mechanism, not an OAuth authorization server. For enterprise user-facing integrations, standards-based OAuth through a central identity provider is a future option. A fixed application identity or managed mTLS may suit service-to-service access. Neither is implemented by this guide.

Windows runtime and repository preparation

Run setup and credential creation under the account intended to execute MCP. Administrator elevation is needed for selected machine configuration tasks, not for every command.

Install and verify prerequisites

```
# Install PowerShell 7 if missing; use the stable x64 MSI if winget is unavailable.
winget install --id Microsoft.PowerShell --exact --source winget
```

Open a new terminal and enter PowerShell 7:

```
pwsh
$PSVersionTable.PSVersion
node --version
hostname
whoami
```

Install dbatools only if not already available:

```
Get-Module -ListAvailable dbatools |
    Select-Object Name,Version,ModuleBase

# If the previous command returned no installed module:
Install-Module dbatools -Scope CurrentUser -Repository PSGallery
```

The observed installed module was `C:\Program Files\WindowsPowerShell\Modules\dbatools\2.8.2`; PowerShell 7 loaded it successfully despite the directory name. No duplicate installation was needed.

Prepare the repository

For an existing checkout:

```
Set-Location C:\Users\jaga_admin\dbatools
git branch --show-current
git pull --ff-only origin development
npm ci
npm run build
npm run refresh-help
npm audit
```

For a new checkout, first clone the intended branch into the chosen folder:

```
git clone --branch development --single-branch `
    https://github.com/jagadd/dbatools.git `
    C:\Users\jaga_admin\dbatools
```

Run commands individually and stop on errors. The generated help index is `generated\dbatools-help.json`; it must agree with the installed module version. Regenerate it after a module update. The POC indexed 731 commands from dbatools 2.8.2. Merely installing dependencies and building does not start HTTPS.

`npm audit` originally reported body-parser and Zod advisories. A reviewed update resolved them, including Zod 4 schema compatibility. The updated lockfile produced zero known npm vulnerabilities when tested. This is a point-in-time result; continue auditing. Do not use `npm audit fix --force` blindly.

Create and configure the POC HTTPS certificate

The POC uses an explicitly trusted self-signed certificate. It is suitable for this controlled test, not a substitute for enterprise certificate management. Its SAN covers `2276259-pyhome`, `localhost`, and `10.10.0.5`. Access through another hostname or address requires a matching certificate and allowed-host entry.

Run the following once in PowerShell 7, elevated if creating the directory requires it. Do not replace an existing private key unintentionally.

```
$certDir = 'C:\McpConfig'
New-Item -ItemType Directory -Path $certDir -Force | Out-Null
if (Test-Path "$certDir\server-key.pem") {
    throw 'A server key already exists; do not overwrite it.'
}

$accountSid = [Security.Principal.WindowsIdentity]::GetCurrent().User.Value
icacls $certDir /inheritance:r /grant:r `
    "*${accountSid}:(OI)(CI)F" `
    '*S-1-5-18:(OI)(CI)F' `
    '*S-1-5-32-544:(OI)(CI)F'
if ($LASTEXITCODE -ne 0) { throw 'Could not protect certificate directory.' }

$rsa = [Security.Cryptography.RSA]::Create(3072)
$request = [Security.Cryptography.X509Certificates.CertificateRequest]::new(
    'CN=2276259-pyhome', $rsa,
    [Security.Cryptography.HashAlgorithmName]::SHA256,
    [Security.Cryptography.RSASignaturePadding]::Pkcs1
)
$names = [Security.Cryptography.X509Certificates.SubjectAlternativeNameBuilder]::new()
$names.AddDnsName('2276259-pyhome')
$names.AddDnsName('localhost')
$names.AddIpAddress([Net.IPAddress]::Parse('10.10.0.5'))
$request.CertificateExtensions.Add($names.Build())

$usages = [Security.Cryptography.OidCollection]::new()
[void]$usages.Add([Security.Cryptography.Oid]::new('1.3.6.1.5.5.7.3.1'))
$request.CertificateExtensions.Add(
    [Security.Cryptography.X509Certificates.X509EnhancedKeyUsageExtension]::new(
        $usages, $false
    )
)
$certificate = $request.CreateSelfSigned(
    [DateTimeOffset]::UtcNow.AddMinutes(-5),
    [DateTimeOffset]::UtcNow.AddDays(90)
)
Set-Content "$certDir\server-cert.pem" `
    -Value $certificate.ExportCertificatePem() -Encoding ascii
```

```
Set-Content "$certDir\server-key.pem" `
    -Value $rsa.ExportPkcs8PrivateKeyPem() -Encoding ascii
$certificate.Dispose()
$rsa.Dispose()
```

`server-cert.pem` is public trust material. `server-key.pem` is private and stays on Windows with restrictive access. Never copy the private key to Linux or Git. The certificate expires 90 days after creation; track its actual expiration.

Authorize the POC account

In an elevated terminal, create the operator group if needed and add the account once. If membership already exists, do not repeat the add operation.

```
if (-not (Get-LocalGroup McpOperators -ErrorAction SilentlyContinue)) {
    New-LocalGroup -Name McpOperators
}
Add-LocalGroupMember -Group McpOperators `
    -Member '2276259-pyhome\jaga_admin'
Get-LocalGroupMember -Group McpOperators
```

For a reader-only account use `McpReaders`. Production execution should use a dedicated account with the required SQL permissions rather than assuming the POC administrator account is the final deployment identity.

Start HTTPS and verify the network path

Windows server environment

In the account's PowerShell 7 console:

```
Set-Location C:\Users\jaga_admin\dbatools
$env:DBATOOLS_TRANSPORT = 'https'
$env:DBATOOLS_HTTPS_HOST = '0.0.0.0'
$env:DBATOOLS_HTTPS_PORT = '8443'
$env:DBATOOLS_HTTPS_ALLOWED_HOSTS = '2276259-pyhome,10.10.0.5,localhost'
$env:DBATOOLS_TLS_CERT_FILE = 'C:\McpConfig\server-cert.pem'
$env:DBATOOLS_TLS_KEY_FILE = 'C:\McpConfig\server-key.pem'
$env:DBATOOLS_CREDENTIAL_STORE = 'C:\McpConfig\credentials'
$env:DBATOOLS_SAFE_MODE = 'true'
node dist/server.js
```

The credential directory can be populated later; profiles are needed when a call uses `credentialRefs`. Keep this terminal running. The default bind address is loopback; `0.0.0.0` explicitly permits network access through the firewall. Allowed hosts are exact names or addresses without schemes, ports, or wildcards.

In a second Windows terminal:

```
curl.exe --cacert C:\McpConfig\server-cert.pem `
    -i https://localhost:8443/mcp
```

Expected: HTTP 401 and `Bearer session token required`. This is a successful unauthenticated transport test, not a failed installation.

Firewall and Azure routing

Permit TCP 8443 only from the Linux app server's appropriate source address or approved management network. Use the actual source address, not this placeholder:

```
New-NetFirewallRule `
    -DisplayName 'MCP HTTPS from Linux app' `
    -Direction Inbound -Action Allow -Protocol TCP `
    -LocalPort 8443 -RemoteAddress 'LINUX_SOURCE_IP'
```

Configure the Azure NSG if its rules would otherwise block that traffic. Ensure VNet routing/peering and Linux egress policy permit the path. Prefer private connectivity; no public endpoint was required for this POC.

Linux trust and connectivity

```
mkdir -p ~/mcp-certs
# Copy ONLY Windows C:\McpConfig\server-cert.pem into this directory.
ip route get 10.10.0.5

curl --cacert ~/mcp-certs/server-cert.pem \
  --connect-timeout 10 -i https://10.10.0.5:8443/mcp
```

The actual Linux test returned HTTP 401 with the expected bearer-session message. This verified routing, TLS verification, and access to the protected endpoint. Do not disable certificate verification to bypass trust or hostname failures.

Login and MCP Inspector testing

Obtain the MCP session on Windows

Keep the server running; use a second PowerShell 7 console in the repository:

```
node dist/server.js login `
    --url 'https://10.10.0.5:8443' `
    --ca 'C:\McpConfig\server-cert.pem'
```

Enter `2276259-pyhome\jaga_admin` and its Windows password (not a Windows Hello PIN, the remote machine's password, or the SQL login password). The helper saves the session token, server origin, and expiration in a protected JSON file. It does not save the Windows password.

For this origin and account, the observed file is:

```
C:\Users\jaga_admin\.dbatools-mcp\sessions\aHR0cHM6Ly8xMC4xMC4wLjU6ODQ0Mw.json
```

Use the path printed by the helper if it differs. Copy only the token privately:

```
$sessionPath = Join-Path $env:USERPROFILE `
    '.dbatools-mcp\sessions\aHR0cHM6Ly8xMC4xMC4wLjU6ODQ0Mw.json'
$session = Get-Content $sessionPath -Raw | ConvertFrom-Json
Set-Clipboard -Value $session.token
[DateTimeOffset]::FromUnixTimeMilliseconds($session.expiresAt).ToLocalTime()
```

`expiresAt` is Unix milliseconds, not a formatted date. The login message also prints an ISO UTC expiration. Protect session files; never paste their contents into chat, screenshots, source code, or issue reports.

Start Inspector on Linux

```
export NODE_EXTRA_CA_CERTS="$HOME/mcp-certs/server-cert.pem"
npx @modelcontextprotocol/inspector
```

This runs a separate MCP test client. Linux does not need PowerShell, SQL Server, or a cloned MCP repository for normal Inspector use. If Inspector was already running, restart it after setting `NODE_EXTRA_CA_CERTS`; Node reads additional CA trust at process startup.

With VS Code Remote SSH, forward Inspector's printed UI port (normally 6274) and open the full printed URL through that forwarded port. Keep Inspector's own local authentication enabled. Its UI/API token is separate from the Windows MCP bearer token and should also be hidden in screenshots.

Add a new connection without deleting unrelated saved connections:

Inspector field	Value
Name	`dbatools-windows`
Transport	HTTP / Streamable HTTP
URL	`https://10.10.0.5:8443/mcp`
Header key	`Authorization`
Header value	`Bearer` followed by the session token

Use exactly one space after Bearer, with no quotes or JSON wrapper. Use custom headers rather than Inspector's OAuth sign-in. Inspector's Node backend must be the process that trusts the certificate and can reach the Windows host.

Environment check

Connect and select `check_dbatools_environment` in Tools, with arguments `{}`. The actual test reported:

```
dbatools 2.8.2 installed and ready
Execution identity: 2276259-pyhome\jaga_admin; no caller impersonation
731 commands indexed; module and index versions match
Safe mode ON; max rows 100; timeout 60 seconds
Persistent execution disabled for HTTPS
```

Inspector is a testing playground, not an LLM agent. In form mode, `commandName` contains only a command name and `parameters` contains its argument object. For full examples below, enable **Edit as JSON** and replace the entire argument document. Do not paste a full JSON request into the commandName text field.

Validate SQL on the MCP host

Direct Windows baseline

```
Get-Service -Name 'MSSQLSERVER','MSSQL$*' -ErrorAction SilentlyContinue |
    Select-Object Name,Status

Import-Module dbatools
$sql = Connect-DbaInstance -SqlInstance localhost `
    -ConnectTimeout 15 -ErrorAction Stop
$sql | Select-Object Name,VersionString,Edition
```

`MSSQLSERVER` identifies the default instance, so `localhost` is correct. A named instance would use `localhost\INSTANCE` in PowerShell or `localhost\\INSTANCE` inside JSON. `Connect-DbaInstance` does **not** accept `EnableException`; the corrected command above was tested successfully.

The local result was server `2276259-pyhome`, version `17.0.1125.2`, Enterprise Developer Edition (64-bit). Windows authentication used the account executing MCP; no separate local SQL password was stored.

Version through MCP Inspector

Invoke `invoke_dbatools_command` with:

```
{
  "commandName": "Get-DbaInstanceProperty",
  "parameters": {
    "SqlInstance": "localhost",
    "InstanceProperty": "VersionString",
    "EnableException": true,
    "ErrorAction": "Stop"
  },
  "selectProperties": [
    "ComputerName", "InstanceName", "SqlInstance",
    "Name", "Value", "PropertyType"
  ]
}
```

The verified result included `Name: VersionString`, `Value: 17.0.1125.2`, and `InstanceName: MSSQLSERVER`. The original `Version` filter returned no rows; that did not establish failed SQL connectivity. Empty results require checking filters and command behavior rather than assuming success or failure.

List and rename databases

Read database state with:

```
{
  "commandName": "Get-DbaDatabase",
  "parameters": {"SqlInstance": "localhost", "EnableException": true},
  "selectProperties": ["Name", "Status", "IsAccessible"]
}
```

A rename from `sales` to `test_sales` was also performed in the POC. Before any new rename, read `get_dbatools_command_help` for `Rename-DbaDatabase`, confirm the intended target, check name conflicts, and account for application connections. The actual invocation pattern was:

```
{
  "commandName": "Rename-DbaDatabase",
  "parameters": {
    "SqlInstance": "localhost",
    "Database": "sales",
    "DatabaseName": "test_sales",
    "EnableException": true,
    "Confirm": false
  },
  "confirm": true
}
```

This example **changes state**; do not repeat a completed rename. The outer `confirm` passes the MCP safety gate. Inner `Confirm: false` suppresses a PowerShell interactive prompt; it is not user authorization. Naming a database does not automatically rename physical files or update application connection strings. Verify the final state with `Get-DbaDatabase -Database test_sales`.

A prior projection of rename output produced `Name: null` and `Status: FULL`. Those fields were unsuitable for that command's return type; they did not mean the rename failed. Choose output fields based on actual command help and verify changes by reading the resulting state.

Connect to the second SQL Server

Target checks

On target `windowstest` (`10.70.1.5`), the default SQL service was running, SQL authentication was enabled, and SQL listened on TCP 1433.

Run these in SSMS on the target to inspect equivalent settings:

```
SELECT
    SERVERPROPERTY('ServerName') AS ServerName,
    SERVERPROPERTY('IsIntegratedSecurityOnly') AS WindowsAuthenticationOnly;

EXEC master.dbo.xp_readerrorlog 0, 1, N'Server is listening on';
```

`WindowsAuthenticationOnly = 0` means SQL authentication is enabled. Changing authentication mode requires separate planning and a SQL restart; no such change was needed here. From the MCP Windows host:

```
Test-NetConnection -ComputerName 10.70.1.5 -Port 1433
```

The observed source was `10.10.0.5`, and `TcpTestSucceeded` was True. This SQL network path is separate from Linux-to-MCP TCP 8443.

Dedicated SQL login and direct connection

In SSMS on the target, create a dedicated SQL-authentication login such as `mcp_poc_reader`, choose a strong password, keep password policy enabled, and disable mandatory password change at first login for this application identity. Start with the `public` server role and grant only the permissions required for the intended checks. Windows local administrator status is not a substitute for SQL login permissions.

Test from the MCP host, entering the password only in the credential prompt:

```
$remoteCredential = Get-Credential -UserName 'mcp_poc_reader'
$remoteSql = Connect-DbaInstance `
    -SqlInstance '10.70.1.5,1433' `
    -SqlCredential $remoteCredential `
    -ConnectTimeout 15 -ErrorAction Stop
$remoteSql | Select-Object Name,VersionString,Edition
```

A SQL TLS trust failure concerns the target SQL certificate, not the MCP HTTPS certificate. Configure the SQL target's certificate and client trust appropriately; do not confuse the two trust chains.

Save a DPAPI credential profile

Run on the MCP host as `2276259-pyhome\jaga_admin`, the execution account:

```
Set-Location C:\Users\jaga_admin\dbatools
$callerSid = [Security.Principal.WindowsIdentity]::GetCurrent().User.Value
.\scripts\save-credential.ps1 `
    -Store 'C:\McpConfig\credentials' `
    -Alias 'windowstest-reader' `
    -AllowedSids $callerSid
```

Enter the remote **SQL login and SQL password**, not the Windows MCP login. The SID above is appropriate because the POC caller and execution account are the same Windows account. For another caller, resolve and authorize that caller's SID deliberately rather than reusing the execution SID.

The script creates a protected `profiles.json` and an encrypted `windowstest-reader.clixml`. DPAPI ties decryption to the executing Windows account on that machine. Profiles cannot simply be copied to another host or service account. Existing aliases are not silently overwritten.

The server resolves aliases, checks allowed SIDs and filesystem ACLs, and passes authorized credentials through stdin to the fixed driver. HTTPS rejects inline credential arguments. STDIO retains legacy inline credential compatibility but references are preferred.

Invoke the remote version check

Ensure the running server has `DBATOOLS_CREDENTIAL_STORE` set. If changing it, restart in the configured terminal, then obtain a new MCP token.

```
{
  "commandName": "Get-DbaInstanceProperty",
  "parameters": {
    "SqlInstance": "10.70.1.5,1433",
    "InstanceProperty": "VersionString",
    "EnableException": true,
    "ErrorAction": "Stop"
  },
  "credentialRefs": {"SqlCredential": "windowstest-reader"},
  "selectProperties": ["SqlInstance", "Name", "Value", "PropertyType"]
}
```

PowerShell still runs on `10.10.0.5`; `SqlInstance` determines the SQL destination. This is remote SQL access, not PowerShell remoting or OS impersonation on the target machine.

Authorization test and serialization fixes

Restricted login versus sysadmin

The following read-only query requires `VIEW SERVER PERFORMANCE STATE` on SQL Server 2022 and later, or `VIEW SERVER STATE` on earlier versions:

```
{
  "commandName": "Invoke-DbaQuery",
  "parameters": {
    "SqlInstance": "10.70.1.5,1433",
    "Database": "master",
    "Query": "SELECT total_physical_memory_kb AS TotalMemoryKB, available_physical_memory_kb AS
        AvailableMemoryKB FROM sys.dm_os_sys_memory;",
    "EnableException": true,
    "ErrorAction": "Stop"
  },
  "credentialRefs": {"SqlCredential": "windowstest-reader"},
  "confirm": true
}
```

The public-only login was denied. After granting sysadmin to that same SQL login, the query succeeded with no profile, MCP token, or configuration change required. HTTPS starts a fresh execution process on each invocation. Remove temporary sysadmin membership after testing and grant only the required permission.

`Invoke-DbaQuery` requires outer `confirm: true` because the generic safety gate classifies the Invoke verb conservatively, even when this particular SQL text is read-only. The gate is not a SQL parser and is not a substitute for SQL permissions.

What failed and what changed

Symptom	Finding and resolution
`parameter.aliases is not iterable`	PowerShell could emit null or scalar aliases; generation and index loading now normalize alias arrays.
Zero rows for the version check	`Version` did not match; verified `VersionString` directly.
Timeout and insufficient memory	Raw .NET/SQL objects were passed directly to JSON serialization. Explicit field selection avoided the problematic graph.
Roughly 568 KB for two query values	A DataRow expanded its Table, column types, and other metadata. The new wrapper extracts only column/value data.

Symptom	Finding and resolution
Underscores rejected in selected columns	Output property validation now allows letters, digits, and underscores after an initial letter; PowerShell expressions remain rejected.
`Name: null` after a successful rename	Output projection did not match the rename return type; verify state using a separate read.

New normalization behavior

The shared driver loads `scripts/serialize-output.ps1` before command execution. The command builder calls `ConvertTo-McpJson` rather than serializing live objects directly. This behavior applies to both HTTPS and STDIO.

- DataRow becomes column/value JSON; DBNull becomes null.
- DataTable and DataSet use bounded table conversion.
- Enums use names and dates use ISO 8601 strings.
- Safe scalar CLR properties are read without traversing complex CLR getters.
- ScriptProperty/AliasProperty getters are not evaluated automatically.
- Credential/SecureString objects are redacted, and resolved secret redaction remains.
- `selectProperties` is still available and applied before normalization.
- Complex nested objects and truncated values use explicit markers.

Bound	Value
Nesting depth	3 levels
Nested collection	100 items
Object properties / row columns	128
Individual string	8,192 characters, then a truncation marker
Aggregate nodes	5,000
Aggregate text including keys	131,072 characters
Normalized JSON	1 MB

Excessive collections, columns, or aggregate output produce actionable limit errors. The existing command row limit (default 100), timeout (default 60 seconds), and driver envelope limits remain. Normalization does not bound memory used inside a dbatools command before it returns.

A serialization error can occur after a modifying command has succeeded. Verify target state before retrying; never automatically repeat a state-changing command solely because output conversion failed.

Verified final remote output

Without `selectProperties`, the query returned:

```
{
  "TotalMemoryKB": 4137684,
  "AvailableMemoryKB": 720560
}
```

This confirmed the fix against the real remote SQL Server. These are host memory figures, not SQL Server allocated memory. Available memory varies between calls.

Operations, restarts, and troubleshooting

What persists

Item	After terminal close or Windows reboot
Certificate and key files	Persist; certificate still expires on its configured date
SQL permissions and Windows group membership	Persist unless changed
DPAPI profile	Persists; requires the same machine/account and usable profile
`$env:` settings in a terminal	Do not persist into unrelated new sessions
User-level environment variables	Persist for new processes under that account
Running Node MCP process	Stops; automatic startup was not configured
Bearer tokens	Invalid after MCP restart; log in again

Persist the credential directory setting once for the current account:

```
[Environment]::SetEnvironmentVariable(
    'DBATOOLS_CREDENTIAL_STORE',
    'C:\McpConfig\credentials',
    'User'
)
```

This stores a path, not a password. New processes inherit it. Other HTTPS settings must also be supplied through a controlled startup configuration or environment. A service/task would need its execution account, environment, user-profile and DPAPI requirements configured explicitly.

Safe update sequence

1. Stop MCP with Ctrl+C in the configured server terminal.
2. Review working-tree changes before pulling; preserve local work deliberately.
3. Pull `development`, build, and test.
4. Regenerate help when the module/index generation changes or dbatools is updated.
5. Restart MCP with the intended environment and identity.
6. Log in again; update Inspector/application secrets and reconnect.

```
git status --short --branch
git pull --ff-only origin development
npm ci
npm run build
```

```
npm test
# If the module or index generator changed:
npm run refresh-help
node dist/server.js
```

Use the same configured terminal or restore all HTTPS environment settings first. Never reset away local work or apply an old stashed lockfile over reviewed updates without inspection.

Troubleshooting reference

Symptom	Check
`pwsh` not found	Install PowerShell 7; open a new terminal or use its full path.
Git says not a repository	Change to the MCP directory containing package.json and .git.
Linux timeout/refused on 8443	MCP process, bind address, Windows firewall, NSG, routes.
HTTPS certificate failure	Correct public certificate, SAN/address match, expiration, Node startup trust.
HTTP 401	Full local username, password, fresh token, server restart, correct Bearer header.
HTTP 403	Allowed host/origin, group/role, and credential-profile SID authorization.
SQL login or SQL TLS failure	Target authentication, grants, listener, and SQL certificate trust separately.
Old token still used by application	Update backend secret and restart/reload the backend; do not restart MCP afterward.
Same validation error after editing	Inspect newest tools/call arguments; replace the complete JSON, remove stale fields.
Huge output or serialization limits	Current wrapper version, actual return type, narrower properties/rows.

Origins are checked when supplied. General browser CORS support is not implemented; use Inspector’s backend or the application backend rather than calling MCP directly from frontend JavaScript.

Claude Agent SDK application handoff

Intended application architecture

```
React chat / structured request UI
          |
          v
Express backend + Claude Agent SDK
          |                 |
          | model access    | MCP tool calls
          v                 v
Claude through          Windows dbatools MCP
Microsoft Foundry            |
                             v
                    Local or remote SQL Server
```

Claude Code is a development tool used to build the application. The Claude Agent SDK is the backend agent framework. Microsoft Foundry supplies model access with its own endpoint/model/authentication settings. The backend, not the model endpoint or browser, establishes the MCP connection and forwards tool results.

The separate application's described React/Express/SSE architecture supports streaming assistant text and tool activity. Proposed ITSM-style requests can provide structured targets, actions, reasons, and action-specific approval. That enhancement is not a claim that the MCP repository implements an ITSM platform.

Backend settings

Adapt these names to the application's actual configuration loader:

```
DBATOOLS_MCP_URL=https://10.10.0.5:8443/mcp
DBATOOLS_MCP_TOKEN=<fresh-token-from-Windows-login>
```

Set certificate trust **before starting Node.js**, not merely in dotenv after Node has started:

```
export NODE_EXTRA_CA_CERTS="$HOME/mcp-certs/server-cert.pem"
# Start the backend using its own documented command.
```

An Agent SDK MCP connection has this shape:

```
mcpServers: {
  dbatools: {
    type: "http",
    url: process.env.DBATOOLS_MCP_URL!,
    headers: {
      Authorization: `Bearer ${process.env.DBATOOLS_MCP_TOKEN!}`,
```

```
    },
  },
}
```

Do not expose Foundry credentials, MCP tokens, SQL passwords, or private keys to the frontend. Keep the server's safe mode enabled and enforce permissions in the backend. A broad UI "Allow changes" toggle should not replace approval of the specific target/action. The MCP confirmation flag alone is not an approval system.

Application target	Tool configuration
Local SQL on MCP host, CI `10.10.0.5`	`SqlInstance: localhost`, Windows execution identity
Remote SQL, CI `10.70.1.5`	`SqlInstance: 10.70.1.5,1433`, credential alias `windowstest-reader`

The application should map authorized target IDs to server-side configuration, discover commands, read help, select useful output, and verify changes. Do not allow a browser to choose arbitrary MCP endpoints or credentials. SQL permission errors should be reported; the agent must not automatically elevate a login.

An agent gateway was considered unnecessary for this single-application POC. If an existing platform supplies one, it can onboard the same endpoint provided it supports custom bearer headers and this certificate trust configuration.

Verification record and implementation map

Notable implementation commits

Commit	Change
`caac6dbe7`	Windows-authenticated HTTPS and credential isolation
`ee8d5d3b4`	Local Windows SID auditing and cleanup of starting sessions
`82e436622`	Reviewed dependency advisory fixes and Zod 4 compatibility
`7ae657df3`	PowerShell alias generation/loading normalization
`7c2b17d90`	Bounded serialization, DataRow flattening, underscore selection

Source map

File	Responsibility
`src/server.ts`	CLI entry point and transport selection
`src/server-factory.ts`	Shared four tools, help resources, risk gate, audit
`src/https-server.ts`	TLS endpoint, host/origin checks, authentication, capacity
`src/auth.ts`, `src/login.ts`	Session hashes, expiry, login helper and token file
`src/credentials.ts`	Alias/SID authorization, credential validation, redaction
`src/tool-registry.ts`	Safe command-script generation and output selection
`src/powershell.ts`, `src/powershell-session.ts`	One-shot and persistent lifecycle/timeouts
`scripts/run-command.ps1`	Fixed stdin driver and response framing
`scripts/serialize-output.ps1`	Bounded conversion to plain JSON
`scripts/authenticate-windows.ps1`	Local Windows logon and group checks
`scripts/save-credential.ps1`	Interactive DPAPI profile creation and ACLs
`scripts/refresh-help.ps1`	Generate command metadata from installed dbatools
`docs/https.md`	Reference setup and current runtime behavior

File	Responsibility
`AGENTS.md`	Development-only contributor context and working rules

Automated and live validation

At `7c2b17d90`, 18 automated tests passed locally with real PowerShell serialization fixtures, type checking passed, and package dry-run included the new serializer. Tests cover TLS/HTTP behavior, roles, sessions, stdin/secret handling, credential policy, concurrency isolation, STDIO parity, DataRow conversion, null/date values, cycles, unsafe getters, string/collection bounds, and output-name validation.

Windows CI additionally passed native logon, account separation, DPAPI, ACL enforcement, persistent queue/recovery, and repeated persistent DataRow conversion. It uses synthetic data and disposable accounts, not the deployed SQL instances.

CI evidence: https://github.com/jagadd/dbatools/actions/runs/40857215367

Live POC evidence is the user's Windows/Linux/Inspector results described above. The final remote query without projection returned only TotalMemoryKB and AvailableMemoryKB, validating the actual deployment behavior beyond fixtures.

Rebuild this PDF

The Markdown file beside this PDF is its editable source. On a documentation build machine with Python 3, Pandoc, Chromium, and the Python `playwright` package installed:

```
python3 scripts/build-deployment-guide.py
```

Use `--no-sandbox` only if the isolated documentation build environment requires it. These tools are not server runtime dependencies. Review generated PDF layout after edits and commit the Markdown source and PDF together on `development`.